Henrik Ejersbo Jensen, Kim G. Larsen, Arne Skou; 1996

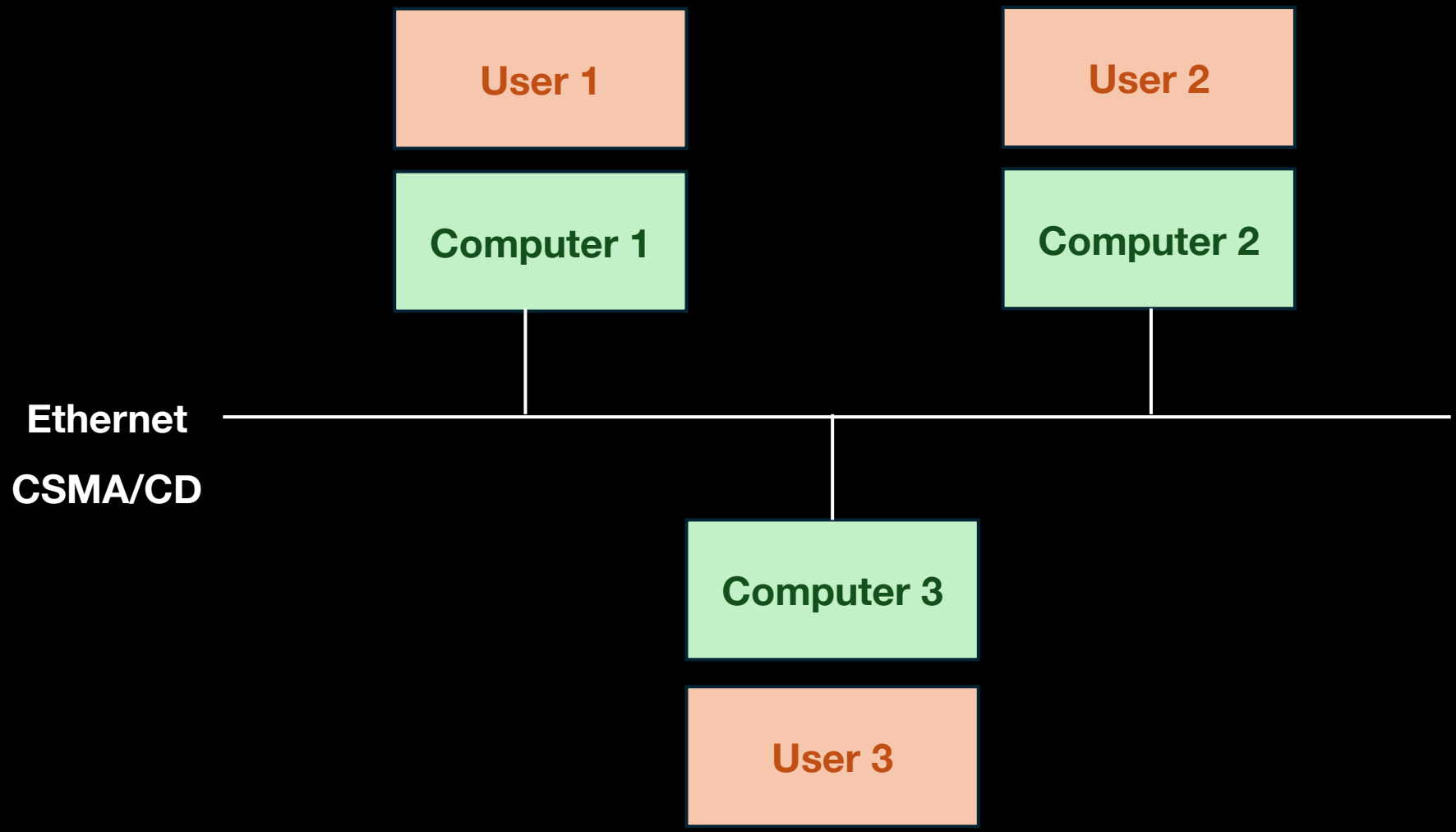
Modelling a Collision Avoidance Protocol using UPPAAL

Aviral Sharma, Darryl David

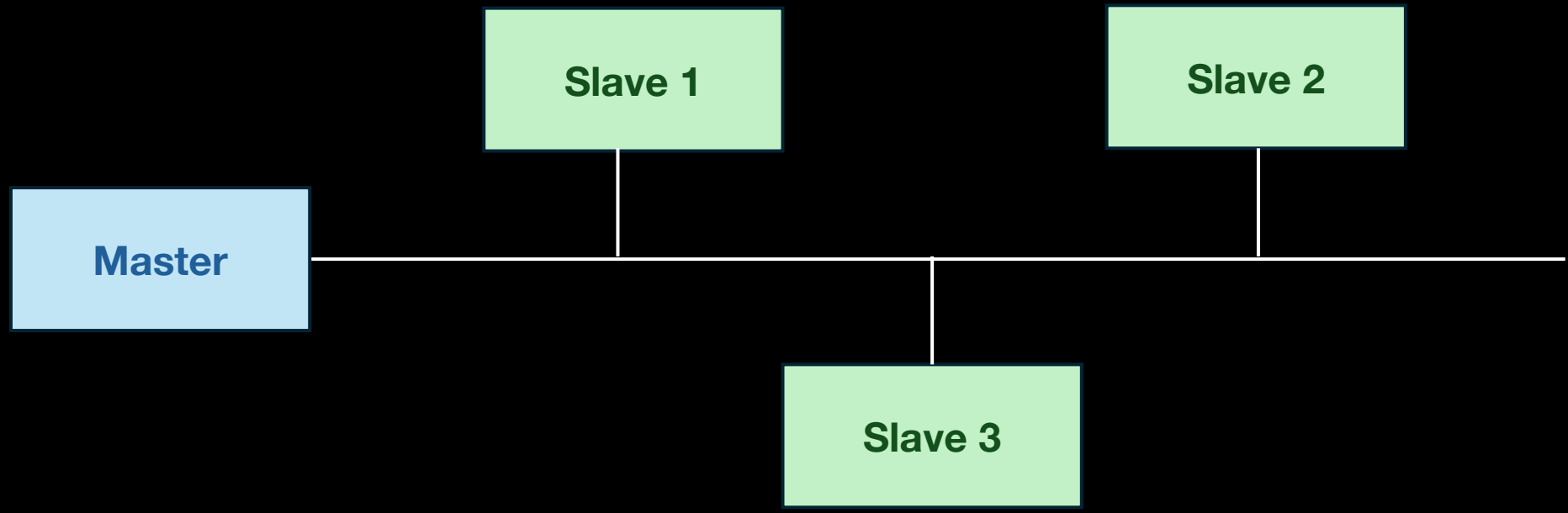
Motive
Solution
Tool

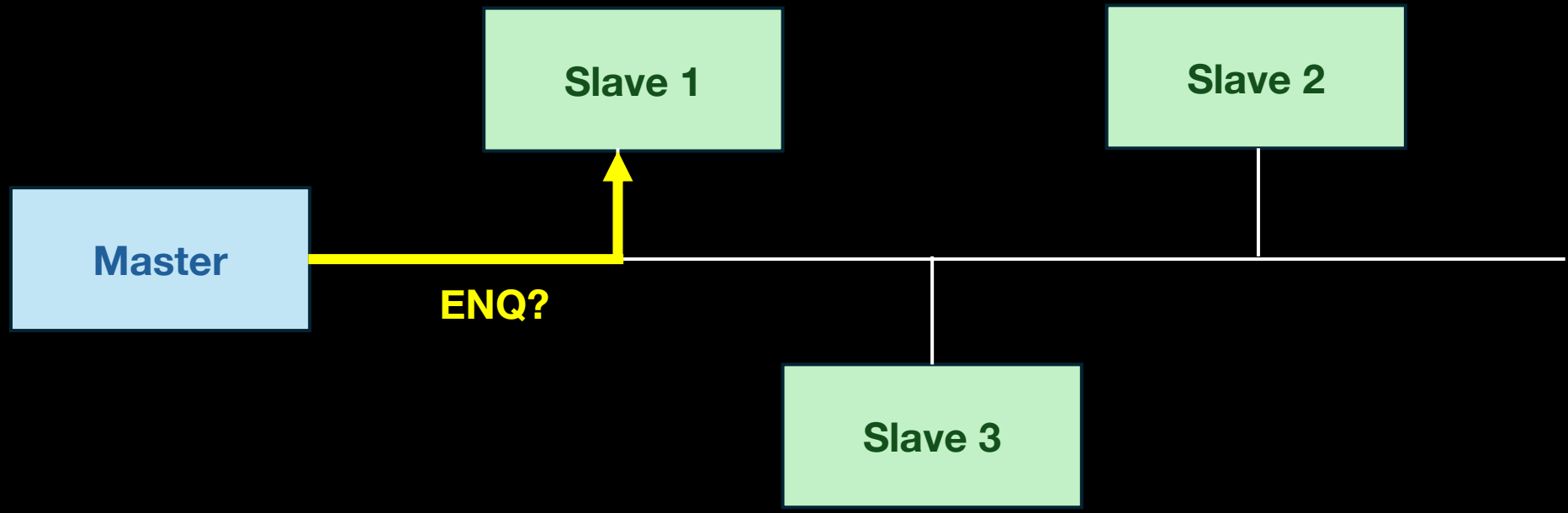
Model

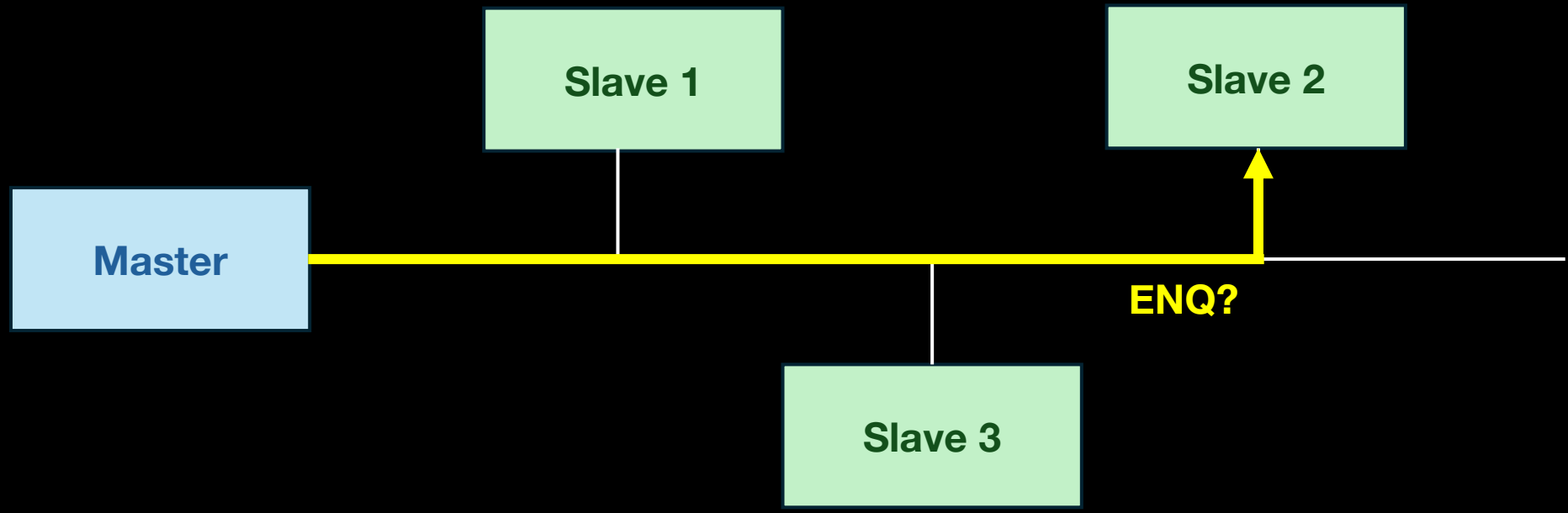
Problem



The absence of collisions isn’t guaranteed

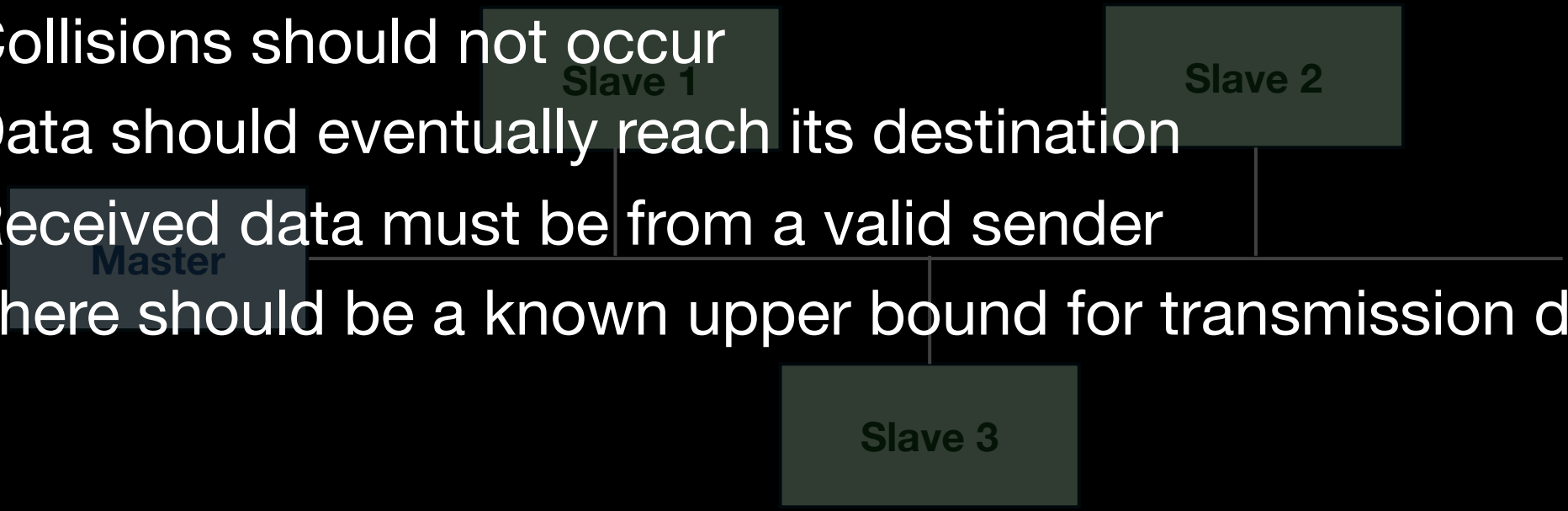

Slave 1
Slave 2
Master
Slave 3


Slave 1
Slave 2
Master
ENQ?
Slave 3


Slave 1
Slave 2
Master
ENQ?
Slave 3

Requirements

- Collisions should not occur
- Data should eventually reach its destination
- Received data must be from a valid sender
- There should be a known upper bound for transmission delay



Tools used to model this system

- SPIN
- UPPAAL

Why UPPAAL

- Timed Automata
- Model Checker with on-the-fly verification
- Diagnostic traces
- Property Language

STL Logic

- l_φ is unreachable in $(T_\varphi \mid A_1 \mid \ldots \mid A_n)$ iff $(A_1 \mid \ldots \mid A_n) \models \varphi$
- Model-checking STL reduces to reachability.
- l_φ is a bad state in a test automata T_φ
- A_i are automata representing our system

Automata

Master
Round-robin enquiry + timer

Medium
Broadcast via committed nodes

Slave (×3)
Receive/forward/ignore data

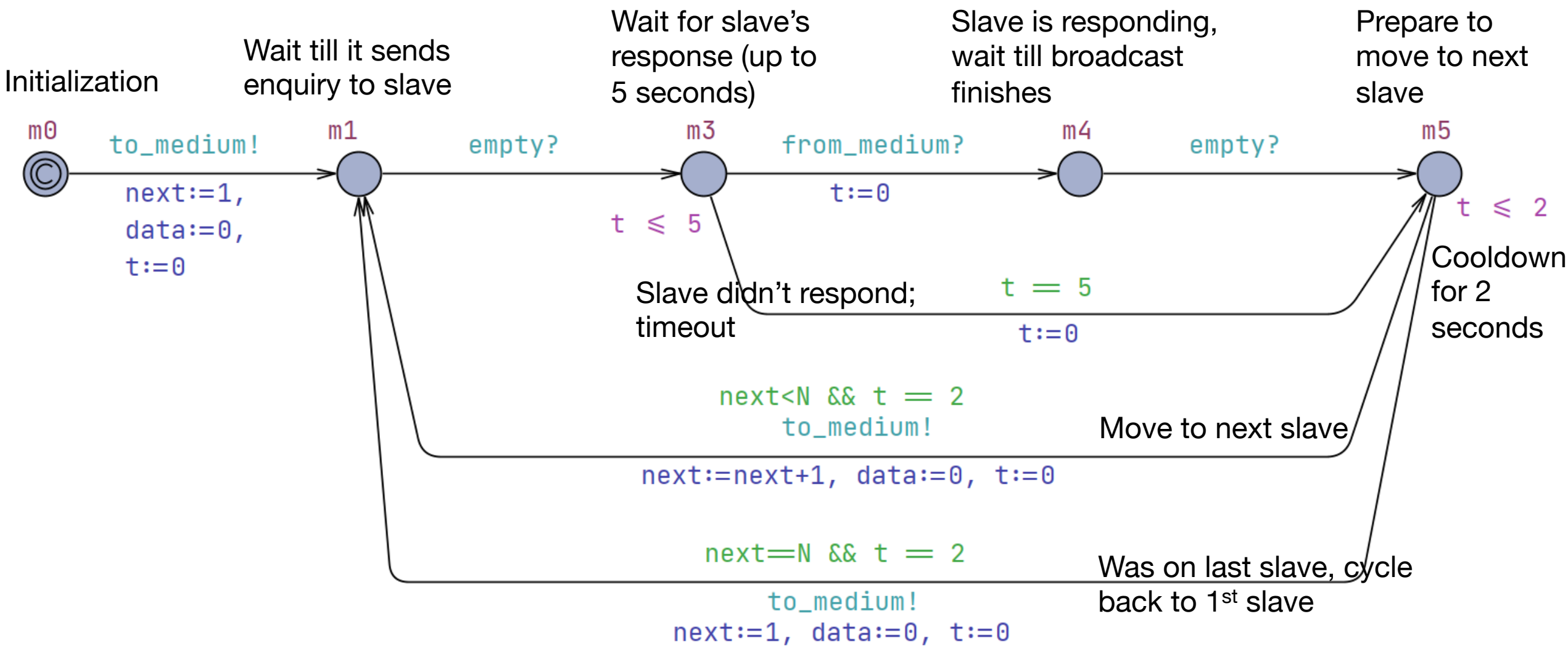
User (×3)
Send/receive/probe options

+ some automata for testing

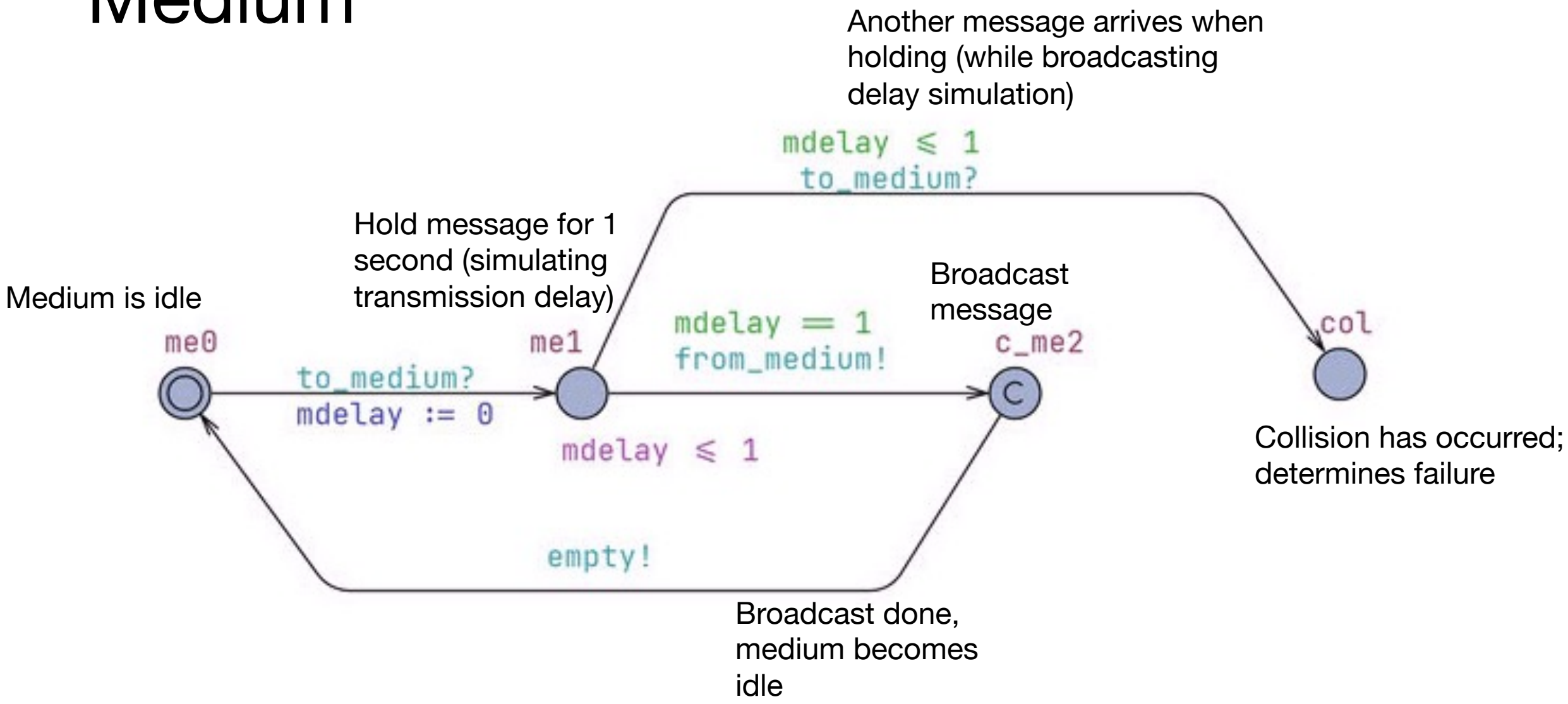
Motive
Solution
Tool

Model

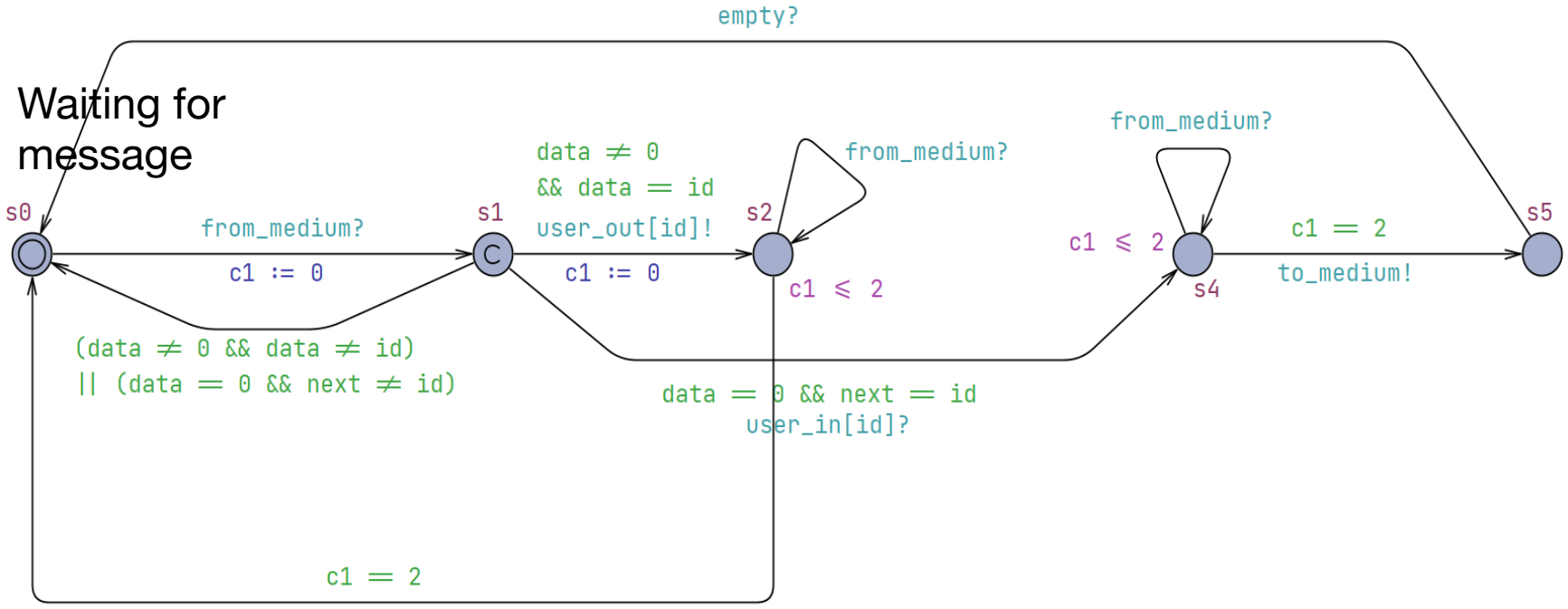
Master

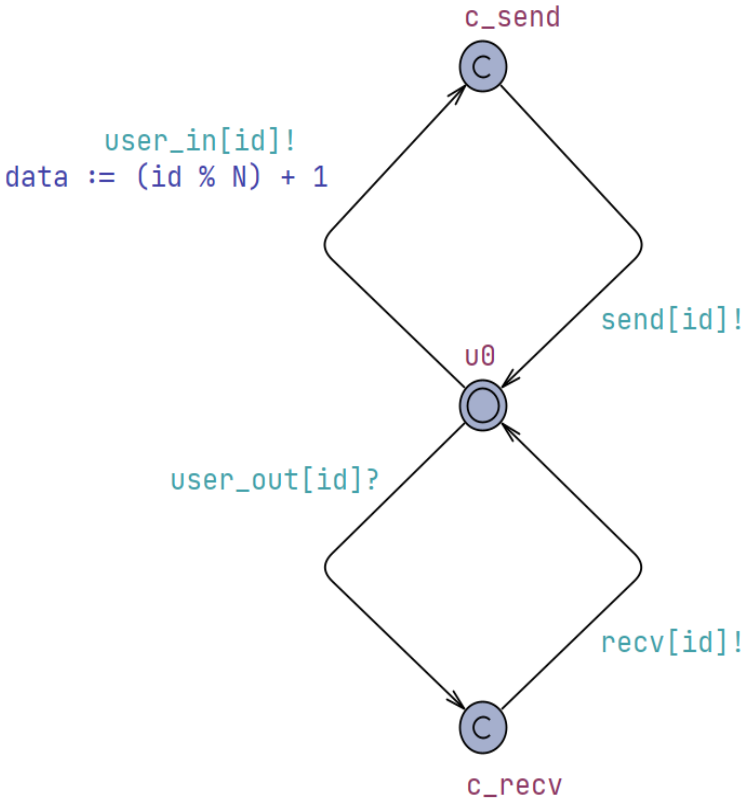

Initialization
Wait till it sends enquiry to slave
Wait for slave's response (up to 5 seconds)
Slave is responding, wait till broadcast finishes
Prepare to move to next slave
m0
to_medium!
next:=1,
data:=0,
t:=0
m1
empty?
m3
t ⩽ 5
from_medium?
t:=0
m4
empty?
m5
t ⩽ 2
Cooldown for 2 seconds
Slave didn't respond; timeout
t == 5
t:=0
next<N && t == 2
to_medium!
next:=next+1, data:=0, t:=0
Move to next slave
next==N && t == 2
to_medium!
next:=1, data:=0, t:=0
Was on last slave, cycle back to 1st slave

Medium

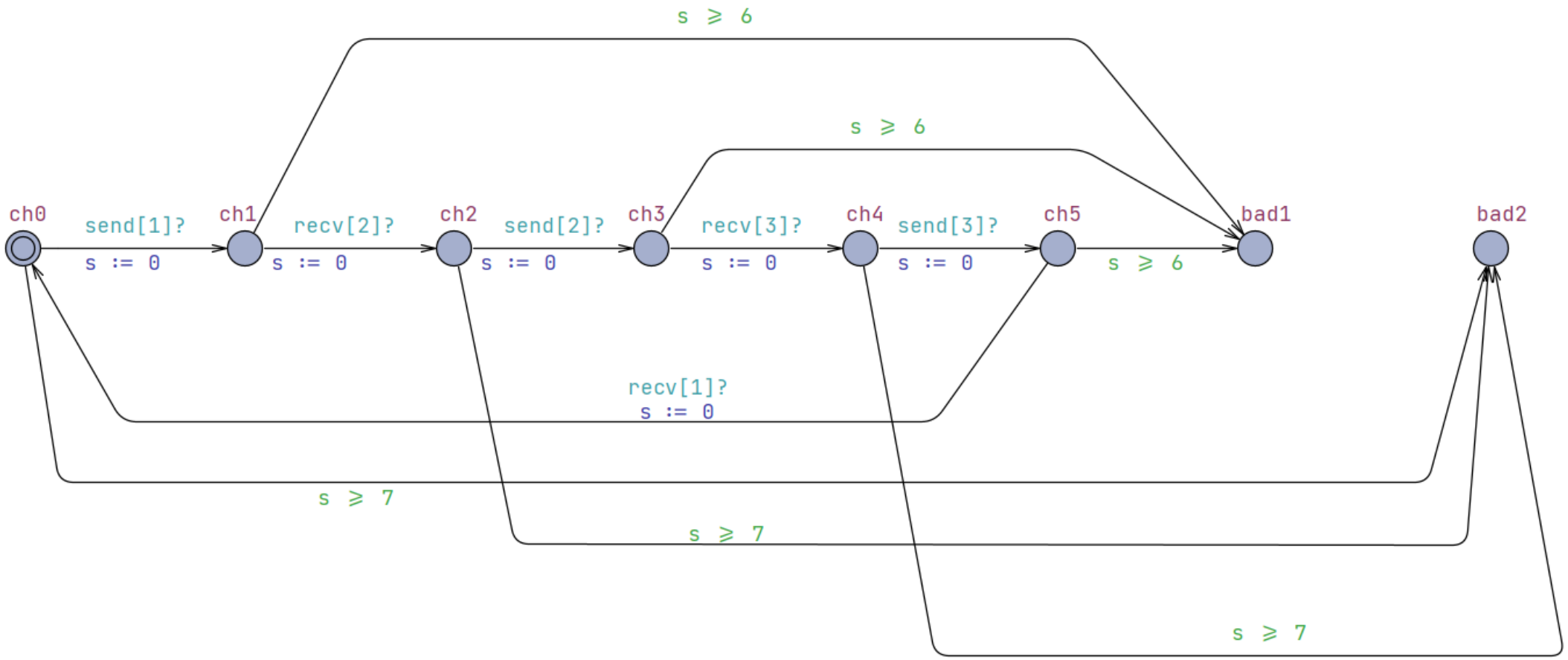

Another message arrives when holding (while broadcasting delay simulation)
mdelay ⩽ 1
to_medium?
Hold message for 1 second (simulating transmission delay)
Broadcast message
Medium is idle
mdelay == 1
from_medium!
me0
me1
c_me2
col
to_medium?
mdelay := 0
mdelay ⩽ 1
empty!
Collision has occurred; determines failure
Broadcast done, medium becomes idle

Slave and User

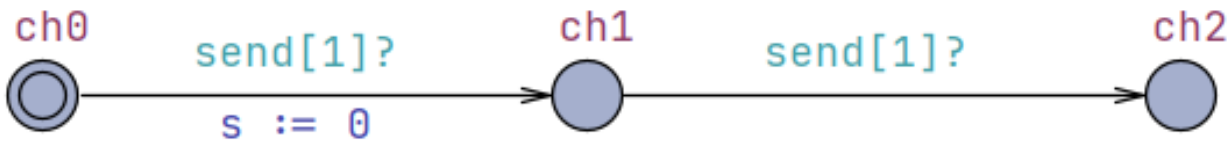




Test Automata 1



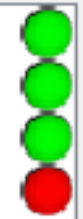
Test Automata 2



Editor | Symbolic Simulator | Concrete Simulator | Verifier

Overview

```
A[](not Medium1.col)
A[] not (Observer1.bad1 or Observer1.bad2)
E<> (Observer2.ch2 and Observer2.s <= 18)
E<> (Observer2.ch2 and Observer2.s <= 17)
```



Check

Insert above

Insert below

Remove

Comments

Clear results

Query

Comment

Editor | Symbolic Simulator | Concrete Simulator | Verifier

Enabled Transitions

recv[2]: User2 →

Reset | Next

Simulation Trace

recv[2]: User2 → Observer1

(m3, me0, s4, s2, s0, u0, u0, u0, ch2, ch1)

Slave2 →

(m3, me0, s4, s0, s0, u0, u0, u0, ch2, ch1)

to_medium: Slave1 → Medium1

(m3, me1, s5, s0, s0, u0, u0, u0, ch2, ch1)

from_medium: Medium1 → Master1, Slave2, Slave3

(m4, c_me2, s5, s1, s1, u0, u0, u0, ch2, ch1)

Slave3 →

(m4, c_me2, s5, s1, s0, u0, u0, u0, ch2, ch1)

empty: Medium1 → Master1, Slave1

(m5, me0, s0, s1, s0, u0, u0, u0, ch2, ch1)

user_out[2]: Slave2 → User2

(m5, me0, s0, s2, s0, u0, c_recv, u0, ch2, ch1)

Prev | Next | Replay

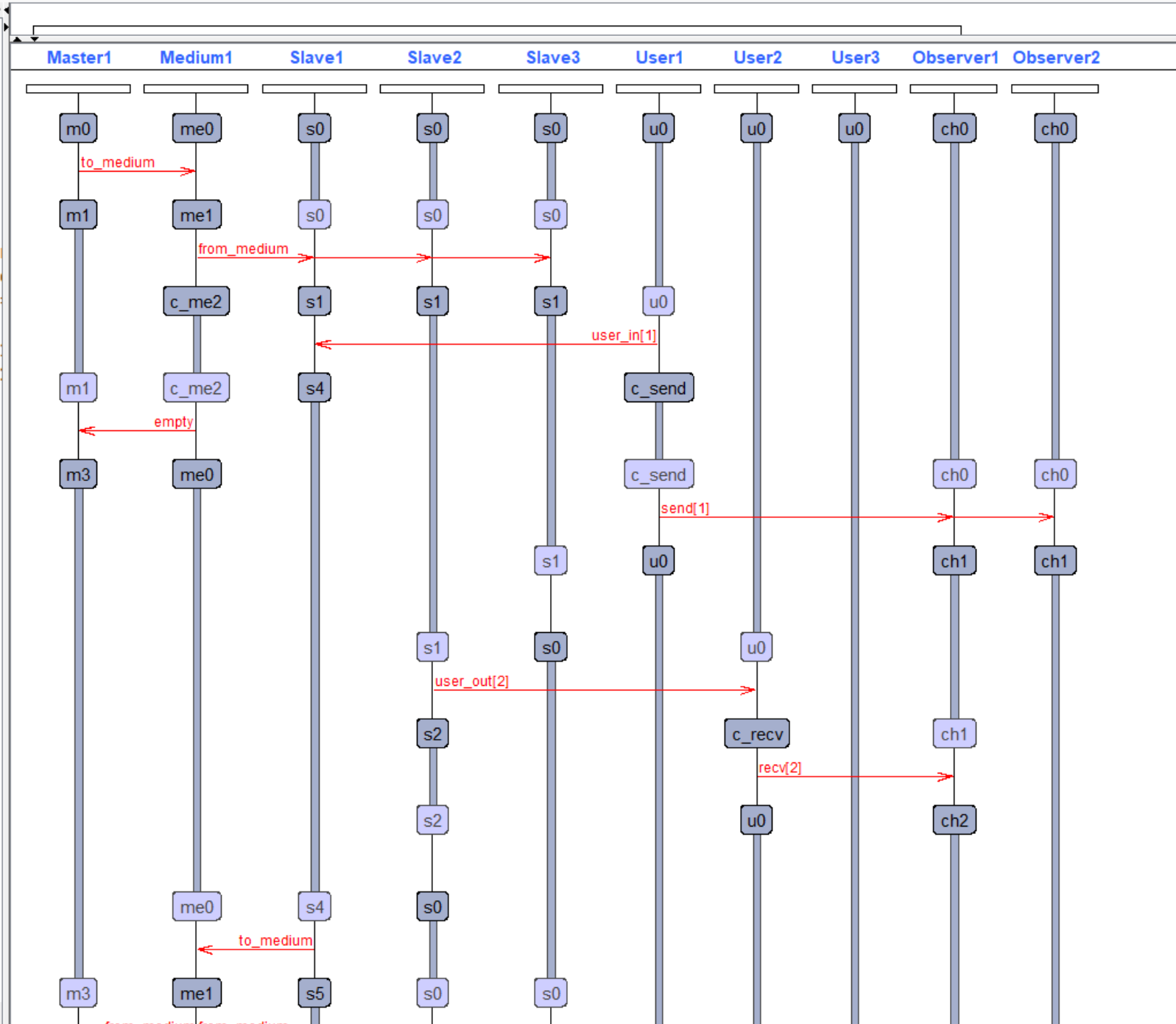
Open | Save | Random

Slow — Fast

Globals

Constraints

- Master1.t = 0
- Medium1.mdelay = 1
- Slave1.c1 = 3
- Slave2.c1 = 0
- Slave3.c1 = 0
- Observer1.s = 3
- Observer2.s = 3
- Medium1.mdelay - Maste
- Slave1.c1 - Medium1.md
- Slave1.c1 - Slave2.c1
- Slave2.c1 = Slave3.c1
- Observer1.s - Slave3.c
- Observer1.s = Observer



Essentially,

- No collisions when timeout >= 3 seconds
- Cycle takes at least 18 seconds

Hence, UPPAAL has verified safety and bounded liveness of this Collision Avoidance Protocol.

Future: Parallelism to let Master reduce round trip to less than each slave’s delay.

Motive
Solution
Tool

Model

Modelling a Collision Avoidance Protocol using UPPAAL